



BILDUNG. FREUDE INKLUSIVE.

Webdesign im IT-L@B

Einführung in HTML und CSS – Modul 16 „Float“

Überarbeitet: 01.10.2025

von Dipl.-Ing. (FH) Robert Schoblick, M.Sc.



Kärntner Berufsförderungsinstitut GmbH
Bahnhofstraße 44
9020 Klagenfurt am Wörthersee
T. +43 (0)5 78 78
info@bfi-kaernten.at

www.bfi-kaernten.at



Die beste
Zeit für
Weiterbildung
ist JETZT!

LESEN (!!!!) Sie dieses Skript aufmerksam durch!!!

Arbeitsauftrag:

1. **Lesen** Sie das Skript **vollständig**!
2. **Programmieren** Sie die vorgestellten Beispiele – auch in eigenen Varianten – nach. Vermeiden Sie es, fremde Programmskripte zu kopieren!
3. **Bearbeiten** Sie den am Ende der Lektion gestellten Arbeitsauftrag!
4. **Testen** Sie die Webseite auf verschiedenen Webbrowsern!

Arbeiten Sie NICHT mit ChatGPT oder anderen KI-Systemen!!!!

Kalkulierte Bearbeitungszeit

Die Aufgabe ist so gestaltet, dass Sie einschließlich der Lektüre dieser Lektion, dem Ausprobieren der Beispiele und eventuell eigene Recherchen im Internet maximal **8 Stunden Arbeitszeit** benötigen sollten. Wenn Sie Probleme (z.B. Schwierigkeiten mit dem Verständnis der deutschen Sprache) haben sollten, wenden Sie sich bitte an Ihre Trainerin oder Ihren Trainer.

Für die Abgabe der Aufgaben gelten folgende Regeln:

Offizielle Lehrlinge des IT-L@B:

Lehrlinge des IT-@B verfügen über einen Zugang zum Lernsystem Moodle. Rufen Sie den folgenden Kurs auf:

20E 55 - User Experience - IT-L@B Villach

Geben Sie die Arbeit im dafür vorgesehenen Abgabefeld der Aufgabe **fristgerecht** ab.

Praktikantinnen und Praktikanten:

Praktikantinnen und Praktikanten haben noch keinen Zugang zum Moodle-System. Sie geben die Aufgabe bitte in Ihrem **persönlichen Ordner** auf dem NAS („\\itlabvi\\public“) ab. Richten Sie dafür bitte ein Unterverzeichnis „web“ ein. Das Speichern auf einem anderen Verzeichnis – z.B.: auf einem lokalen PC – gilt als NICHT abgegeben!

Inhalt

LESEN (!!!!) Sie dieses Skript aufmerksam durch!!!	2
Arbeitsauftrag:	2
Kalkulierte Bearbeitungszeit	2
Für die Abgabe der Aufgaben gelten folgende Regeln:	2
Offizielle Lehrlinge des IT-L@B:.....	2
Praktikantinnen und Praktikanten:	2
Die CSS-Eigenschaft „float“	4
Die CSS-Eigenschaft „clear“	5
Mehrspaltiges Seitenlayout mit „float“	6
Zentraler Bereich mit Außenabständen	8
Zentraler Bereich mit Innenabständen	9
Den Footer richtig platzieren	10
Übungsaufgaben	12

Die CSS-Eigenschaft „float“

Es sei einmal an das erste Modul dieses Workshops erinnert: der Einstieg in CSS. Ziel dieses Moduls war es, vor allem absoluten Neueinsteigern ein Gefühl für das Programmieren von Webseiten zu vermitteln, aber auch, festzustellen, wo bereits Personen mit einiger Grunderfahrung fachlich „abzuholen“ sind. Ein – wenngleich sehr kurz angesprochenes – Thema war die Ausrichtung eines Bildes – das Logo – im Element <header>. Das Logo sollte neben einer Höhendeklaration vor allem auf der rechten Seite im Kopf der Webseite ausgerichtet werden. Dazu wurde im Modul 1 die CSS-Eigenschaft „float“ verwendet. „float“ kennt nur zwei relevante Werte: right, left (und none). Der letztere Wert soll an dieser Stelle nicht interessieren. Dieser schaltet „float“ für das Element aus, als wäre die Eigenschaft gar nicht verwendet worden.

Zur Erinnerung: Der HTML-Code (Ausschnitt des Elements <header>) sieht im Modul wie folgt aus:

```

<h1>Willkommen im IT-L@B des bfi-Kärnten</h1>
</header>
```

Die Bilddatei bringt mit 987px x 600px durchaus eine „sportliche“ Größe mit. Diese wird mithilfe der folgenden CSS-Regel in der Darstellung auf der Seite deutlich reduziert.

```
header img {
    width: 80px;
}
```

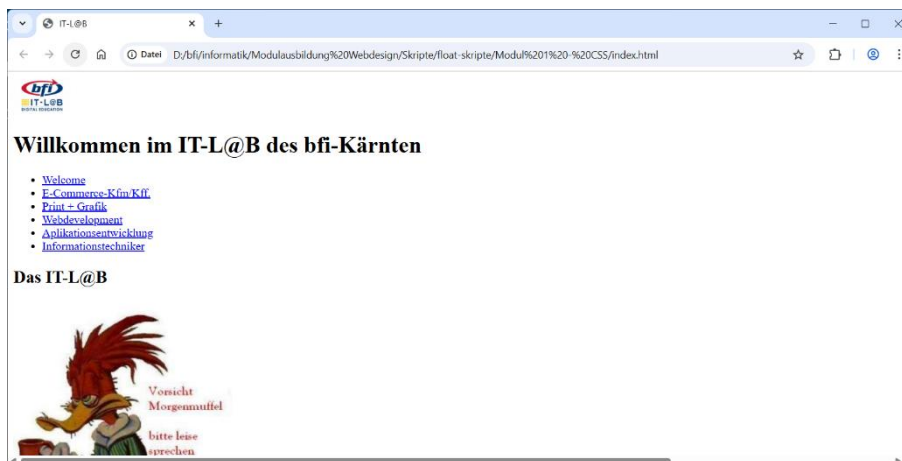


Abbildung 1: "Wunderschön" - Nur sollte das Logo nicht über der Überschrift, sondern rechts davon im Header stehen.

Der CSS-Regelblock wird um eine Zeile erweitert, welche das Bild an der rechten Seite ausrichtet:

```
header img {
    width: 80px;
    float: right;
}
```

Den Begriff „float“ kann man mit „schwimmen“, „treiben“ oder „umlaufen“ übersetzen. Die Bedeutung „umlaufen“ passt im Webdesign am besten, denn das hier formatierte Bild umläuft nun den übrigen Inhalt – hier: den des Headers – auf der rechten Seite. Dabei verdrängt das „gefloatede“ Element die übrigen Inhalte nach links – bzw. nach rechts bei „float: right;“.

Hinweis:

Eine gute Metapher für „float“ ist ein Stein, der in ein Becken mit Wasser gestellt wird. Wird der Stein auf der rechten Seite platziert (= „float: right;“), fließt das Wasser natürlich links an diesem vorbei.

Die CSS-Eigenschaft „clear“

Das folgende Beispiel – der HTML-Code des Beispiels aus Modul 1 ist etwas gekürzt und das Logo soll mithilfe von CSS in der Darstellung auf eine Größe von 500px erweitert werden – zeigt den Einfluss des „Floatens“ über die Grenzen der Elemente hinweg:

Erwartungsgemäß kommt das Logo nun wieder mit einer beachtlichen Größe (width: 500px;) daher, doch wie deutlich zu sehen ist, steht es auch rechts von der (Menü)-Liste und ragt sogar in den Text des ersten <article>-Elements hinein, der dem Bild bereitwillig Platz einräumt.

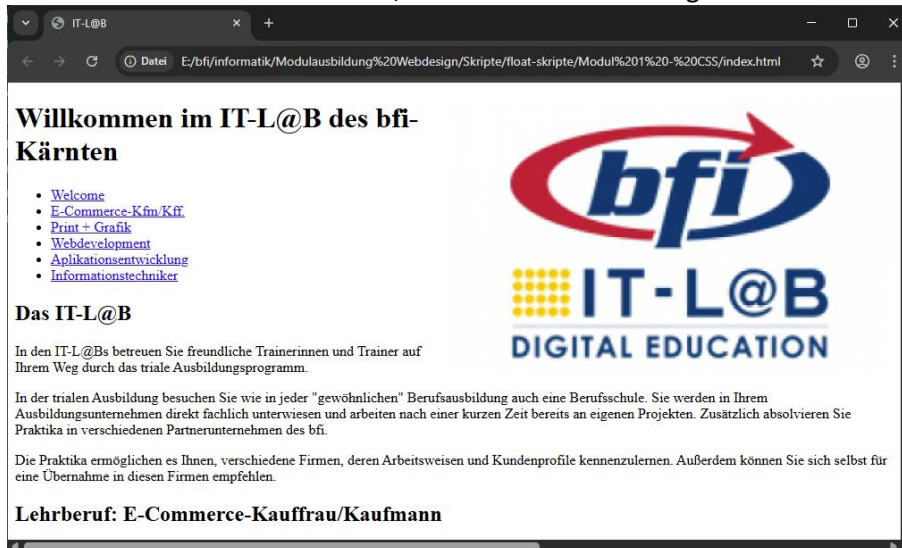


Abbildung 2: Das Bild auf der rechten Seite wurde mit „float: right;“ ausgerichtet. Wegen dessen Größe überragt es allerdings die weiteren Elemente. Anstatt diese zu überdecken, wirkt das Floating auch auf die nachfolgenden Elemente und verschiebt deren seitlichen Grenzen.

Der Effekt, wie ihn Abbildung 2 zeigt, lässt sich vermeiden, indem das Floaten in den Formaten der nachfolgenden Elemente gestoppt wird. Hierzu kommt die CSS-Eigenschaft „clear“ zum Einsatz, welche die Werte „none“, „left“, „right“ oder „both“ zugewiesen werden können.

„clear: none;“ wird man bestenfalls in Ausnahmefällen verwenden, um entgegen einer allgemeinen Festlegung im CSS-Code für ein Element individuell das Floaten der vorherigen Elemente zu erlauben. Geläufiger sind die folgenden Regeln:

- **clear: left;** - Diese Regel deaktiviert die Wirksamkeit eines zuvor mit „float: left;“ deklarierten Umflusses. Ein Umfluss „rechts“ bleibt dagegen auch an diesem Element aktiv!
- **clear: right;** - Diese Regel deaktiviert die Wirksamkeit eines zuvor mit „float: right;“ deklarierten Umflusses. Ein Umfluss „links“ bleibt dagegen auch an diesem Element aktiv!
- **clear: both;** - Es werden grundsätzlich alle Umflüsse (rechts und links) ab einschließlich dieses Element deaktiviert.

Das in Abbildung 2 gezeigte Problem, dass ein Bild in den Darstellungsbereich des nachfolgenden Artikels hinein reicht, kann also mit den folgenden Formaten beseitigt werden. Allerdings rutschen die nachfolgenden Inhalte entsprechend der Höhe des Bildes nach unten:

```
header img {
    width: 500px;
    float: right;
}

article {
    clear: right;
}
```

Die Wirkung zeigt das folgende Bild:



Abbildung 3: Die Regel "clear: right;" wurde in das Format des Elements <article> geschrieben. Die Artikel beginnen also stets unterhalb des Bildes. Dieses umfließt ab dem Beginn des Artikels nicht mehr die nachfolgenden Inhalte.

Ein weiteres Beispiel – hier wird die Eigenschaft „clear“ bereits im Format des <nav>-Elements verwendet – zeigt den Effekt noch deutlicher:

```
nav {
    clear: right;
}
```

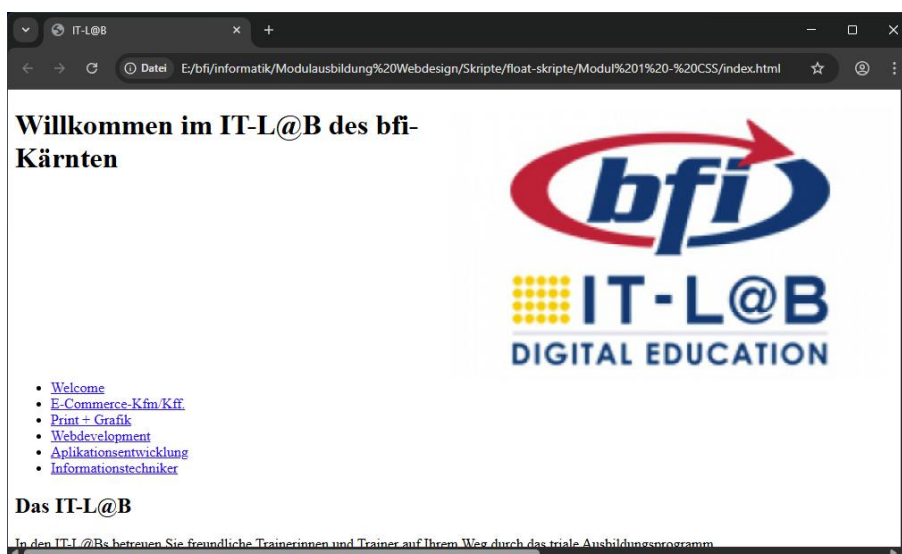


Abbildung 4: Hier wurde die CSS-Eigenschaft "clear" bereits im Format des <nav>-Containers verwendet. Das gesamte Menü rutscht somit unter das Bild. Der Platz auf der linken Seite ist in diesem Beispiel ungenutzt.

Mehrspaltiges Seitenlayout mit „float“

Die Eigenschaft „float“ kann durchaus auch verwendet werden, um ein zwei- oder dreispaltiges Seitenlayout zu entwickeln. Tatsächlich war dies vor Zeiten der Flex- und Grid-Technologie, die verschiedene Vorteile bieten, gängige Praxis.

Die Aufteilung einer Webseite in mehrere Spalten mithilfe von „float“ ist im Grunde genommen sehr einfach, fordert aber auch Kompromissbereitschaft des Programmierers (und dessen Auftraggeber) in der Gestaltung der Seiten ab. Es können recht gute Ergebnisse erzielt werden, wenn auf verschiedene Hintergründe in den Spalten sowie auf Rahmenlinien zwischen den Spalten verzichtet wird.

Um ein mehrspaltiges Layout mithilfe der CSS-Eigenschaft *float* zu erzeugen, bedarf es mehrerer Schritte:

- Definition der rechten und linken Umläufe (*float: right;* und *float:left;*) in den Formaten der betreffenden Elemente, welche die äußeren Spalten verkörpern sollen.
- Festlegung der linken und rechten Innenabstände des umlaufenden Elementes. Die Inhalte der Spalten sollen optisch durch einen kleinen Abstand voneinander abgegrenzt werden.
- Aufhebung der Umläufe für das erste nachfolgende Element, das wieder über die volle Seitenbreite dargestellt werden soll.

Im Beispiel sollen, wie gesehen, der Header und der Footer über die volle Breite der Seite verlaufen. Links soll ein Navigationsbereich, rechts eine zusätzliche Randspalte gesetzt werden. Im mittleren Bereich soll der Kernartikel seinen Platz finden. Um die Wirkung der Formate besser sichtbar zu machen, werden den Elementen ausnahmsweise verschiedene Hintergrundfarben zugewiesen.

```
nav {  
    float: left;  
    width: 200px;  
    background-color: red;  
}
```

Der Navigationsbereich <nav> soll auf der linken Seite des Fensters platziert werden. Dies wird mit der Eigenschaft

```
float: left;
```

festgelegt. Sie besagt, dass das Element (hier: <nav>) den regulären Textfluss auf der *linken* Seite umläuft. Diese Deklaration allein reicht jedoch noch nicht aus, denn es ist noch die Breite der „Navigationsspalte“ festzulegen, die im Beispiel 200 Pixel betragen soll. Dies wird mit der Eigenschaft

```
width: 200px;
```

erreicht.

Noch einmal sei darauf hingewiesen:

Die hier im Beispiel gesetzte Hintergrundfarbe dient lediglich Anschauungszwecken. Sie sollte in echten Projekten nicht an dieser Stelle verwendet werden.

Entsprechend wird mit dem Element <aside> verfahren, das den regulären Textfluss auf der *rechten* Seite umlaufen soll. Dies wird mit der CSS-Eigenschaft „*float: right;*“ erreicht. Auch diese „Spalte“ muss in der Breite definiert werden und soll eine Breite von 150 Bildpunkte haben.

```
aside {  
    float: right;  
    width: 150px;  
    background-color: green;  
}
```

Betrachtet man das Ergebnis im Browser-Fenster, so wird dies noch nicht zufriedenstellend sein. Zwar entsteht eine Anordnung, die einem dreispaltigen Layout ähnlich erscheint, jedoch unterfließt der Haupttext – dieser befindet sich im Element <article> – den linken, rot unterlegten Bereich bis an den Rand des Viewports. Wäre der Inhalt des <aside>-Elements (grüner Hintergrund) kürzer, so würde Gleiches auch auf der rechten Seite passieren. Darüber hinaus ist der Footer (gelb unterlegter Bereich in Abbildung 5) nicht unterhalb aller drei Spalten und auch nicht über die volle Seitenbreite angeordnet.

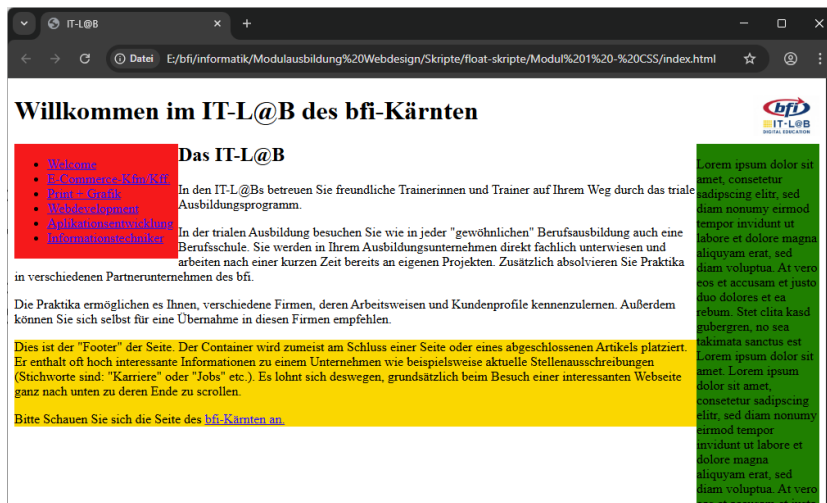


Abbildung 5: Die CSS-Eigenschaften `float: left;` (roter Bereich) und `float: right;` (grüner Bereich) funktionieren in Verbindung mit den jeweiligen Breiten-Deklarationen einwandfrei. Dennoch ist das Ergebnis bei Weitem noch unbefriedigend. Es sind Ergänzungen erforderlich!

Zunächst soll der zentrale Bereich im Element `<article>` so angeordnet werden, dass dieser nicht in eine der beiden Randspalten hineinragen kann. Wichtig ist dabei, dass es sich um ein Blockelement handelt, da sonst die Abstandsdefinitionen nicht die gewünschte Wirkung¹ erzielen. `<article>` gehört per Definition bereits zu den Blockelementen. Eine spezielle Deklaration ist deswegen nicht nötig. Im Zweifelsfall kann die Block-Eigenschaft jedoch mit

```
display: block;
```

erzwungen werden.

Damit der Eindruck eines (Drei-)Spaltenlayouts entsteht, müssen zusätzlich Randabstände für das Element `<article>` festgelegt werden. Nach dem CSS-Box-Modell können dies sowohl Innen- (Padding) als auch Außenabstände (Margin) sein. Beides ist möglich, jedoch zeigen die Ergebnisse unterschiedliche Wirkungen.

Zentraler Bereich mit Außenabständen

Im ersten Fall soll der Bereich `<article>` mit Außenabständen² (Margin) versehen werden. Die Wahl der linken und rechten Abstände ist dabei nicht willkürlich, sondern sie richtet sich nach der zuvor festgelegten Breite der Außenspalten (hier: `<nav>` mit 200px und `<aside>` mit 150 Bildpunkten). Zu den Breiten der Außenbereiche wurden jeweils 10 Bildpunkte zugegeben, um einen sichtbaren Abstand zwischen den Inhalten zu gewährleisten. Dieser „Offset“ kann je nach Bedarf entsprechend variiert werden. Auf einen Innenabstand soll hier verzichtet werden. Die entsprechende CSS-Regel zeigt der folgende Skriptausschnitt und Abbildung 6 das Ergebnis im Browserfenster.

```
article {
    margin: 0px 205px 0px 155px;
    padding: 0px;
    background-color: silver;
}
```

¹ Bei einem Inline-Element würde die Deklaration eines Innenabstandes (Padding) bewirken, dass der Beginn des Textes in der ERSTEN Zeile verschoben wird. In der letzten Zeile würde es einen inneren Abstand zum nächsten Element innerhalb der Zeile geben. Zwischen der ersten und der letzten Zeile wäre kein Effekt zu beobachten.

² Im Beispiel werden die Abstandsdeklarationen mit vier Werten angegeben. Zur Erinnerung: Der erste Wert entspricht „top“, dann im Uhrzeigersinn weiter „right“, „bottom“ und „left“.

Hinweis:

Anstelle der einzeiligen Deklaration der Außenabstände (die Regel sieht folgende Belegung der Parameter vor: „margin: top right bottom left;“ können die Abstände natürlich auch mit den CSS-Eigenschaften „margin-left“ und „margin-right“ festgelegt werden.

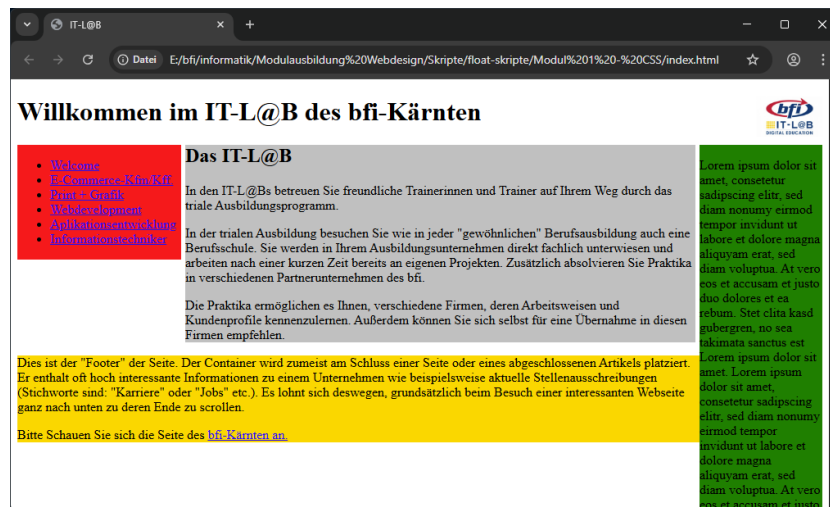


Abbildung 6: Schon ein „ganz netter“ Versuch: Dem Bereich <article> wurden rechts und links ein Außenabstand (Margin) zugewiesen. Der Inhalt ragt bis an die Grenzen des Elementes heran, überschneidet aber nicht mehr die beiden Randbereiche. Der Fußbereich ist jedoch noch immer nicht wunschgemäß ausgerichtet.

Zentraler Bereich mit Innenabständen

Alternativ zur „Spaltenbildung“ mithilfe von Außenabständen können auf der rechten und linken Seite auch Innenabstände (*padding*) deklariert werden. Dies hat zur Folge, dass mögliche Hintergrundfarben des mittleren Bereiches (hier: <article>) über die volle Breite des Browserfensters laufen und eventuell kürzere Randspalten – in diesem Fall den Abschnitt <nav>, der durch eine rote Hintergrundfarbe gekennzeichnet ist – umlaufen werden. Das Ergebnis zeigt Abbildung 7. Der zugehörige CSS-Regelblock sieht wie folgt aus:

```
article {
padding: 0px 205px 0px 155px;
margin: 0px;
background-color: silver;
}
```



Abbildung 7: Statt eines Außenabstandes wurden in diesem Beispiel für den `<article>` Innenabstände festgelegt. Der Inhalt passt sich tatsächlich an die Spaltenstruktur an, während jedoch der Hintergrund nach wie vor den Platz links unterhalb des `<nav>`-Elementes (roter Bereich) einnimmt.

Den Footer richtig platzieren

Die spaltenweise Anordnung der drei Elemente – hier: `<nav>`, `<article>` und `<aside>` ist gelungen, jedoch stört die Position des Footers das gesamte Erscheinungsbild enorm. Auch andere nachfolgende Elemente würden sich diesem Schema unterwerfen und sich wie hier der Fußbereich in das Bild einordnen. Die Aufgabe ist es jedoch, den Footer *unterhalb* der dreispaltigen Ansicht und über die volle Bildschirmbreite zu platzieren.

Ursache für dieses nicht zufriedenstellende Ergebnis ist der nach wie vor wirksame Umlauf der Elemente `<nav>` und `<aside>` durch die Float-Eigenschaft. Float erzwingt einen Umlauf um das betreffende Element, ganz gleich, welche und wie viele Elemente folgen.

Damit (hier) der Footer nicht um die Randelemente herum „ließen“ kann und unterhalb der drei Spalten aus `<nav>`, `<article>` und `<aside>` erscheint, muss der Umfluss für dieses Element abgeschaltet werden. Dies bewirkt die CSS-Eigenschaft „*clear*“ (vgl. Seite 5), wobei gezielt das rechte oder linke Floating oder auch beide Umläufe gleichzeitig deaktiviert werden können. Letzteres (beide, „*clear: left; / clear: right;*“ bzw. „*clear: both;*“) soll in diesem Beispiel geschehen, damit der Footer – ganz gleich wie lang die Inhalte von `<nav>` und `<aside>` sind – die volle Fensterbreite einnehmen kann.

```
footer {
    clear: both;
}
```

Da mit dem Footer der Umlauf unterbrochen wird, müssen diesem eventuell nachfolgende Elemente nicht mit einer „*clear*“-Eigenschaft formatiert werden. Abbildung 8 zeigt das nun den Anforderungen entsprechende Ergebnis. In der Praxis würde dieses Ergebnis noch immer nicht ansprechend sein, denn die mehrfarbigen und unterschiedlich langen Hintergründe stören. Hier offenbart sich eine Schwäche dieses Verfahrens, denn ein Mehrspaltenlayout mit der CSS-Eigenschaft *float* ist nur dann sinnvoll, wenn auf individuelle Hintergründe sowie auf Begrenzungslinien (Border) verzichtet wird.

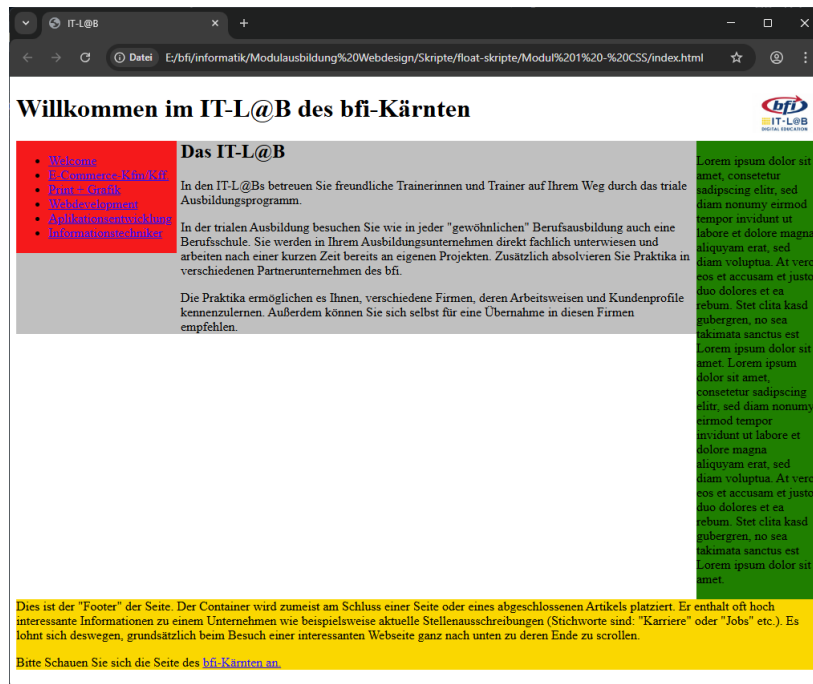


Abbildung 8: Ein dreispaltiges Layout mit „gefloateten“ Blockelementen. Zwar ist durch geschickte CSS-Programmierung ein mehrspaltiges Layout möglich, die verschiedenen Hintergrundfarben zeigen jedoch, dass gestalterische Kompromisse eingegangen werden müssen.

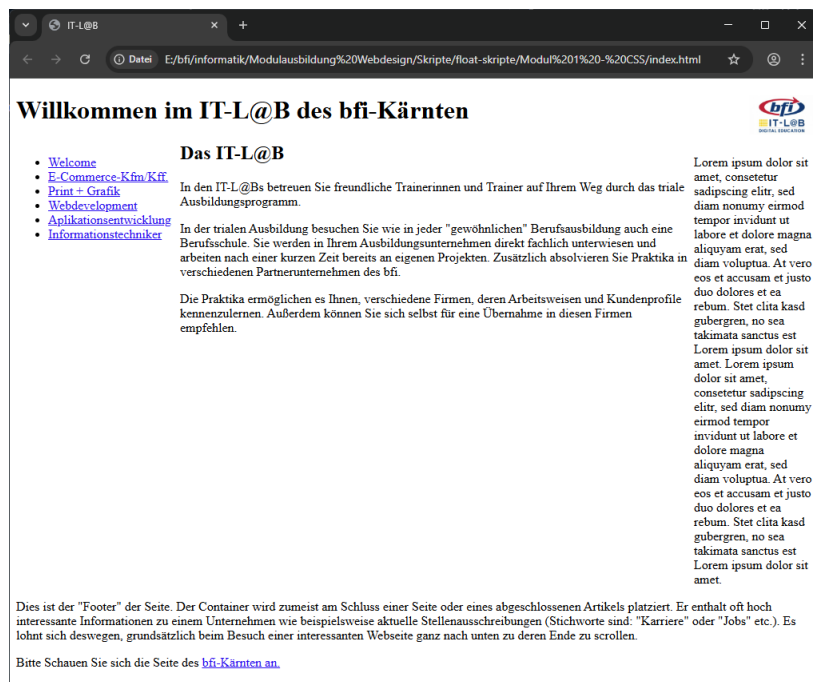


Abbildung 9: Verzichtet man auf Hintergründe in den Spalten-Elementen, so ist das Mehrspaltenlayout mit „float“ durchaus praxistauglich.

Übungsaufgaben

- Aufgabe 16.1: Sehen Sie sich den HTML-Code aus dem Modul 1 noch einmal genauer an. In verschiedenen Artikeln finden Sie zwei Bilder. Verwenden Sie *strukturelle CSS-Pseudoklassen* (**KEINE** Klassenattribute), um die Bilder abwechselnd rechts und links im Textfluss anzuordnen. Benutzen Sie die „Float“-Technologie.
- Aufgabe 16.2: Programmieren Sie die CSS-Datei `format.css` aus dem Modul 1 so, dass bei einem Display mit einer Hochkant-Ausrichtung (z.B.: Smartphone) ein einspaltiges, bei einem Monitor in horizontaler Ausrichtung ein zweispaltiges Layout entsteht. Verwenden Sie ausschließlich die Float-Technologie! Header und Footer sollen jeweils die volle Bildschirmbreite einnehmen! Schlagen Sie ggfs. Auch im Modul 13 (Medienabfragen) noch einmal nach!
- Aufgabe 16.3: Was müssen Sie tun, damit im zweispaltigen Layout das Menü (senkrechte Button) rechts neben dem Hauptteil (Main) angezeigt wird? Auch hier soll nur die Float-Technologie verwendet werden.